

Módulo 5: Superficie del Servidor

T14: Dependencias Vulnerables

Carlos Rodríguez Contreras · GitHub: karrocon

 *Log4Shell, npm audit y por qué tus dependencias son tu superficie de ataque*

Objetivos

- Entender el concepto de CVE y CVSS Score
- Usar `npm audit` y Snyk para detectar dependencias vulnerables en VulnApp
- Entender el concepto de SBOM (Software Bill of Materials)
- Configurar Dependabot en GitHub para alertas automáticas

Tu código vs. tus dependencias

Tu código: ~5.000 líneas
node_modules/: ~500.000 líneas

El 97% del código de tu app no lo has escrito tú.

Cada dependencia es una puerta de entrada potencial.

Log4Shell (CVE-2021-44228)

Una sola dependencia Java (log4j) afectó a millones de servidores.

CVSS: **10.0** (máximo).

Explotación: solo con un string en un campo de texto.

CVE y CVSS – el sistema de clasificación

CVE (Common Vulnerabilities and Exposures): identificador único

CVE-2021-44228 → Log4Shell
CVE-2023-44487 → HTTP/2 Rapid Reset

CVSS (Common Vulnerability Scoring System): severidad 0.0–10.0

Rango	Severidad	Acción
9.0–10.0	Critical	Parchear inmediatamente
7.0–8.9	High	Parchear en días
4.0–6.9	Medium	Planificar parche
0.1–3.9	Low	Evaluar riesgo

npm audit — tu primera línea de defensa

```
cd VulnApp  
npm audit
```

Severity	high	
Package	jsonwebtoken	
Vulnerability	jwt.verify() accepts alg:none	
Fix	Upgrade to >=9.0.0	

```
npm audit fix          # correcciones automáticas (semver compatible)  
npm audit fix --force  # actualizaciones mayores (puede romper)
```

SBOM — Software Bill of Materials

Qué es: inventario completo de todas las dependencias (directas + transitivas) de tu aplicación.

Para qué sirve:

- Saber si usas una librería afectada por un nuevo CVE
- Cumplimiento normativo (FDA, EU Cyber Resilience Act)
- Gestión de licencias

```
# Generar SBOM en formato CycloneDX  
npx @cyclonedx/cyclonedx-npm --output-file sbom.json
```

Dependabot / Renovate — actualizaciones automáticas

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/VulnApp"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 10
```

Resultado: cada semana, si hay actualizaciones de seguridad, Dependabot abre un PR automático.

✓ Combinar con CI (tests automáticos) para merge con confianza.

Supply Chain Attacks – el riesgo emergente

Ataque	Ejemplo real
Typosquatting	<code>lodash</code> → <code>lodashs</code> (paquete malicioso)
Dependency Confusion	Paquete interno suplantado en npm público
Mantenedor comprometido	<code>event-stream</code> (2018) – malware inyectado
Protestware	<code>colors.js</code> / <code>faker.js</code> (2022) – sabotaje del autor

Defensas: lockfile, verificación de integridad (`npm ci`), scoped registries, auditoría regular.

Lockfile — por qué `npm ci` es más seguro que `npm install`

```
npm install    # Resuelve versiones según ^/~ → puede instalar versiones nuevas
npm ci        # Instala EXACTAMENTE lo que dice package-lock.json
```

Aspecto	<code>npm install</code>	<code>npm ci</code>
Resuelve versiones	Sí (puede cambiar)	No (usa lockfile exacto)
Borra node_modules	No	Sí (instalación limpia)
Modifica lockfile	Puede	Nunca
Reproducibilidad	Parcial	Total
Usar en CI/CD	✗	✓

⚠ Sin lockfile, un atacante que compromete una dependencia transitiva puede inyectar código en tu próximo deploy sin que cambies nada.

Lab T14: Auditando dependencias con npm audit

Objetivo: detectar y corregir dependencias vulnerables en VulnApp

Setup: `cd VulnApp`

1. Ejecuta `npm audit` y observa el informe
2. Identifica vulnerabilidades con severidad HIGH o CRITICAL
3. Ejecuta `npm audit fix` para las correcciones automáticas
4. Para las que requieren cambio manual, actualiza la versión en `package.json`

 **Herramientas:** [Snyk](#) · [OSV.dev](#) · [CVE Details](#)